

DiffBio: End-to-End Differentiable Bioinformatics and the Case for Jointly Optimized Preprocessing

Mahdi Shafiei

Avitai Bio, mahdi@avitai.bio

Abstract

Bioinformatics pipelines are built from discrete, non-differentiable stages, and the choices that matter most—normalization, gene selection, and dimensionality reduction—are frozen and tuned in isolation *before* the downstream model ever sees the data. We present **DiffBio**, a JAX/Flax library that reimplements core bioinformatics operations as differentiable relaxations so that an entire analysis workflow can be optimized end to end by gradient descent, and we use it to test a concrete hypothesis: that jointly optimizing the dimensionality reduction with the task model recovers task-relevant structure that a frozen, variance-maximizing projection discards. On cell-type annotation over the Tabula Sapiens atlas, replacing frozen principal component analysis (PCA) with a PCA-initialized learnable projection, trained jointly with the classifier, improves held-out macro-F1 by **+5.2 pp** at an aggressive reduction ($k=10$ components) on a 120,000-cell / 59-type subset, growing to **+10.2 pp** on a 193,000-cell / 87-type subset, with the largest gains (**+13.1 pp**) on rare cell types. The advantage is regime-dependent: it is decisive under aggressive compression, survives a stronger supervised gene-selection baseline (**+5.9 pp**), and narrows to a non-significant **+1.6 pp** when many components are retained ($k=50$)—joint optimization helps precisely when one must compress hard, and is otherwise harmless. Ablations attribute the gain to the learnable projection specifically, and a differentiable “soft- k ” operator recovers a good effective dimensionality without a grid search. The effect then generalizes: across four further pipelines—DNA regulatory- element classification, Perturb-seq perturbation identity, scATAC-seq annotation, and variant calling—and two distinct reduction machineries (linear projection and permutation-invariant read pooling), the same substitution yields **+7.5** to **+16.9 pp** at aggressive reduction, while a no-bottleneck control on the same variant data shows no gain, pinning the mechanism to compression that discards task signal. DiffBio is open source.

1 Introduction

A modern single-cell or genomic analysis is a pipeline: raw measurements pass through quality filtering, normal-

ization, feature (gene) selection, dimensionality reduction, and finally a task model such as a cell-type classifier or a variant caller. Each stage is typically a *discrete, hand-tuned* operation—a hard quality threshold, an **argmax** base call, a fixed number of highly variable genes, a frozen principal-component basis. Two consequences follow. First, discrete operations block gradient flow, so the pipeline cannot be optimized as a whole. Second, and more consequentially, the preprocessing is fit *before*, and independently of, the downstream objective: the projection that a variance-maximizing PCA chooses need not be the projection that best serves cell-type annotation.

This staged, frozen design is near-universal in tools such as Scanpy [1] and is effective, but it leaves a gap. In neighboring fields, making the preprocessing *learnable* and training it jointly with the task has repeatedly paid off: learnable audio frontends outperform fixed mel-filterbanks (LEAF [2], SincNet [3]), differentiable preprocessing search beats fixed default pipelines on tabular data (DiffPrep [4]), and learned augmentation policies beat hand-designed ones [5]. In single-cell genomics, probabilistic models such as scVI/scANVI [6, 7] learn a latent representation jointly with (optionally) the label, but within a single fixed generative model family rather than as a general, composable, differentiable pipeline.

We bring the differentiable-preprocessing paradigm to bioinformatics. Our contributions are:

1. **DiffBio**, an open-source JAX/Flax [8, 9] library that implements bioinformatics operators—alignment, variant calling, single-cell analysis, epigenomics, RNA-seq, normalization, and more—as differentiable modules with a uniform interface, so heterogeneous operators compose into pipelines that train end to end (Section 2).
2. A **joint-optimization formulation** for the single-cell preprocessing-to-annotation pipeline, in which a learnable, PCA-initialized projection is trained together with the annotation model against the label loss, together with a differentiable *soft-k* operator that learns the effective number of components (Section 3).
3. An empirical study establishing, with honest attention to where it does and does *not* hold, that

joint optimization beats frozen PCA for cell-type annotation—by +5.2 to +10.2 *pp* macro-F1 under aggressive reduction, growing with atlas size and concentrated on rare cell types, surviving a stronger supervised baseline, and narrowing to insignificance when reduction is generous—and then **generalizing** it across four further real-data pipelines and a second reduction machinery (permutation-invariant read pooling), with a mechanistic no-bottleneck control that isolates the cause (Section 4).

2 The DiffBio library

DiffBio provides differentiable implementations of core bioinformatics algorithms so that heterogeneous analysis stages compose into a single pipeline trainable by gradient descent. It is the biology-specific operator layer of a JAX/NNX scientific-ML stack (Figure 1), built on four foundation packages: *Datarax* (operator and dataflow contracts), *Artifex* (reusable model-building and optimizer components), *Opifex* (advanced optimization primitives, including the GradNorm multi-loss balancer used below), and *Calibrax* (metrics and regression control), all atop Flax NNX and JAX/XLA for GPU execution.

2.1 Operator abstraction and dataflow

Every operation is an `OperatorModule`: a Flax NNX module carrying a frozen-dataclass configuration, validated at construction, and a single `apply` method that maps a dictionary of named arrays to an updated dictionary (Listing 1). Three conventions make otherwise incompatible operations interoperable and differentiable:

- **Dictionary dataflow.** Operators read the keys they need and forward the rest untouched (e.g. "counts"→"features"→"projection"→"logits"), so a normalizer, a projection, and a classifier chain without bespoke glue.
- **One-hot sequence encoding.** Sequences are (length×alphabet) arrays rather than integer indices, so gradients flow through base identity—a prerequisite for differentiating alignment and variant operators.
- **Temperature-controlled smoothing.** Each discrete decision is a smooth relaxation with a temperature that recovers the hard operation in the limit, giving training an explicit bias/variance control.

Trainable state is declared as `nnx.Param` and fixed buffers (a frozen PCA anchor, running statistics) as `nnx.Variable`. A single `nnx.split(model, nnx.Param)` therefore exposes *exactly* the trainable parameters to `jax.grad`, whether they live in one operator or are scattered across a ten-stage pipeline—the mechanism that makes end-to-end optimization automatic.

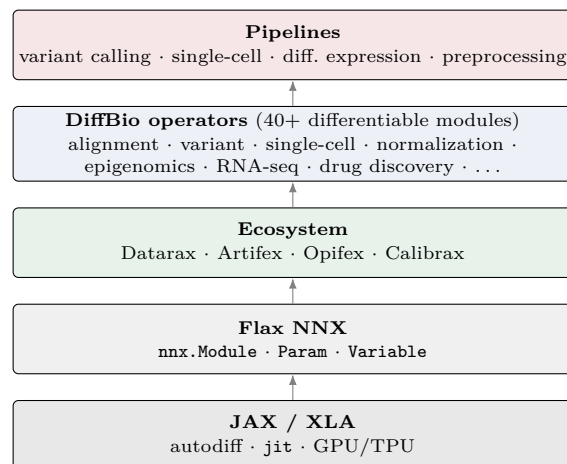


Figure 1: DiffBio architecture. A uniform `OperatorModule` interface lets 40+ differentiable operators across bioinformatics domains compose into end-to-end pipelines, atop the Datarax/Artifex/Opifex/Calibrax ecosystem and JAX/Flax.

2.2 Operator catalog

DiffBio ships 40+ operators across the major bioinformatics domains (Table 1): sequence alignment (a `logsumexp`-relaxed Smith–Waterman [10], profile HMMs, soft multiple alignment), variant calling (a differentiable pileup feeding a CNN caller, after DeepVariant [11], plus copy-number segmentation and quality recalibration), single-cell analysis, epigenomics, RNA-seq, statistical models, drug discovery, genome assembly, and protein/RNA structure. The single-cell suite exercised in this paper—learnable normalization, soft highly-variable-gene selection, differentiable scaling and PCA, and the learnable projection and soft-*k* operators of Section 3—is one column of this catalog; six end-to-end pipelines (variant calling, enhanced variant calling, differential expression, single-cell, perturbation, and preprocessing) are pre-assembled from it.

2.3 Making discrete operations differentiable

Four recurring relaxations replace the discrete primitives that block gradient flow, each recovering its hard counterpart as the temperature $\rightarrow 0$: (i) *logsum-exp* for dynamic-programming maxima (alignment scores, HMM decoding); (ii) *sigmoid gates* for hard thresholds (quality filtering, soft-*k* component selection); (iii) *segment-sum* for counting (pileup depth, feature aggregation), replacing a histogram with a weighted accumulation; and (iv) *Markov-power accumulation* for diffusion operators (imputation, pseudotime), replacing an eigendecomposition—whose gradient is unstable on the degenerate spectra common in single-cell graphs—with repeated sparse matrix products.

Table 1: Representative DiffBio operator domains and the relaxation each relies on.

Domain	Representative operators	Relaxation
Alignment	Smith–Waterman, profile HMM, soft MSA	log-sum-exp
Variant	pileup, CNN caller, CNV segmentation, quality recalibration	segment-sum
Single-cell	normalization, soft HVG, PCA, clustering, batch correction, RNA velocity	sigmoid, power
Normalization	VAE normalizer, UMAP, PHATE, learnable / orthonormal projection, matrix-free PCA, soft- k	power, sigmoid
Epi./RNA-seq	peak calling, chromatin state, splicing PSI, motif discovery	sigmoid
Statistical	HMM, EM quantification, NB-GLM	log-sum-exp
Drug disc.	GNN message passing, fingerprints, AttentiveFP, ADMET	GNN
Structure	protein secondary structure, RNA folding	log-sum-exp
Multi-omics	Hi-C contact, spatial deconvolution/detection	segment-sum

2.4 Composition and end-to-end training

Operators compose through a `CompositeOperatorModule` with a sequential strategy, with lightweight adapters renaming dictionary keys between stages; because composition preserves each child’s parameters, the assembled pipeline is one differentiable graph. DiffBio’s training utilities then optimize it end to end (Listing 2): an AdamW optimizer from the Artifex factory over the `nnx.Param` leaves, deterministic mini-batching, gradient clipping, and—when a preprocessing regularizer must be balanced against the task loss—an Opifex GradNorm balancer that weights objectives by their training rates rather than a hand-tuned coefficient. Calibrax supplies held-out metrics and regression control.

This paper validates the joint-optimization instantiation in depth on single-cell annotation and then across four further pipelines (Section 4); broad per-operator validation of the rest of the catalog is out of scope, and we keep the breadth catalog (Table 1) deliberately separate from the rigorous case studies that follow.

3 Jointly optimized preprocessing

Setup. Consider cell-type annotation. Let $X \in \mathbb{R}^{n \times g}$ be raw counts for n cells over g genes. The standard frozen pipeline applies a fixed transform T (library-

Listing 1: The uniform operator interface: a learnable projection.

```
class LearnableProjection(OperatorModule):
    def __init__(self, config, *, init_loadings, rngs):
        super().__init__(config, rngs=rngs)
        # fixed PCA anchor + zero-init learnable residual
        self.basis = nnx.Variable(init_loadings) # frozen
        self.delta = nnx.Param(jnp.zeros_like(init_loadings))
        self.bias = nnx.Param(jnp.zeros(config.n_components))

    def apply(self, data, state, metadata, *):
        projection = self.basis[...] + self.delta[...]
        z = data["features"] @ projection + self.bias[...]
        return {**data, "projection": z}, state, metadata
```

Listing 2: Composing operators into a pipeline and training every stage’s parameters end to end.

```
pipeline = CompositeOperatorModule(
    CompositeOperatorConfig(
        strategy=CompositionStrategy.SEQUENTIAL,
        operators=[LearnableNormalization(...), SoftHVG(...),
                    DifferentiableScaler(...),
                    LearnableProjection(..., init_loadings=V),
                    LinearEmbeddingProbe(...)]),
    rngs=rngs)

# one optimizer trains every nnx.Param across all stages
opt = nnx.Optimizer(pipeline, create_optimizer(cfg), wrt=nnx.Param)

for batch in loader:
    grads = nnx.grad(loss_fn)(pipeline, batch)
    opt.update(pipeline, grads)
```

size normalization, log, highly-variable-gene selection, z -scoring) and a variance-maximizing projection $V \in \mathbb{R}^{h \times k}$ (the top k PCA loadings on the h selected genes), then trains only a classifier f_ϕ on the frozen features $Z = T(X)V$:

$$\min_{\phi} \mathcal{L}(f_{\phi}(T(X)V), y). \quad (1)$$

V is chosen to maximize retained *variance*, which need not align with the directions that separate cell types.

Joint objective. We instead make the projection learnable, initialized at the PCA solution, and train it jointly with the classifier:

$$\min_{\theta, \phi} \mathcal{L}(f_{\phi}(T(X)(V + \Delta_{\theta})), y), \quad (2)$$

where Δ_{θ} is a residual initialized to zero, so training *starts* exactly at the frozen-PCA baseline and can only improve on the training objective before adapting (the LEAF initialization recipe [2]). Because V is a fixed, data-independent matrix, this objective is well defined under mini-batch stochastic gradient descent, unlike re-estimating PCA per mini-batch (Section 4.1).

Statistics are frozen, not re-estimated per batch. T ’s global statistics (gene means/variances) and the PCA anchor V are fit *once* on the training split and held as constants (`stop_gradient`), following the batch-normalization pattern [12] and DiffPrep’s fit-once trans-

form [4]. This keeps the objective stationary and, critically, removes the activation-memory blowup of back-propagating through the full-batch pipeline: only the projection and classifier are on the gradient tape, so training an atlas-scale batch fits comfortably on a single GPU.

Learning the dimension. The number of components k is itself a design choice. We additionally provide a differentiable *soft-k* operator that keeps the PCA directions fixed but learns a cumulative-variance coverage threshold, softly gating the eigenvalue-ranked components; it recovers a hard truncation as its temperature vanishes and, trained against the loss, selects an effective dimensionality without a grid search.

4 Experiments

4.1 Setup

Data. We use the Tabula Sapiens human cell atlas [13]: a 120,000-cell subset spanning six tissues with 59 cell types, and a broader 193,000-cell subset spanning all ten available tissues with 87 cell types. Cells are split 80/20 train/test, stratified by cell type; all preprocessing statistics are fit on the training split only.

Arms. Both arms share the frozen transform T and differ only in the reduction: (i) **Frozen PCA**—fixed top- k loadings, only the classifier trains (the standard pipeline); (ii) **Joint projection**—the PCA-initialized learnable projection $V + \Delta_\theta$, trained jointly. Both feed an identical two-layer MLP classifier and are trained by deterministic mini-batch AdamW (batch 4096, 100 epochs). This isolates the effect of *learning the reduction* from gene selection and classifier capacity. We report held-out macro-F1 and balanced accuracy, mean $\pm$ s.d. over three seeds, with a rare-cell-type breakdown (classes in the lowest quartile of training frequency).

Baselines and scope. Our controlled comparison is frozen PCA with an identical classifier, which isolates the reduction. Comparison against jointly-trained generative models (scVI/scANVI [6, 7]) and other feature-selection schemes is important and left to future work (Section 5); we also compare against a stronger *supervised* gene-selection baseline below.

4.2 Joint optimization beats frozen PCA

At an aggressive reduction to $k=10$ components, the joint projection improves held-out macro-F1 from 0.730 ± 0.005 to 0.783 ± 0.003 on the 120k/59-type atlas, a $+5.2 pp$ gain with all three seeds positive; balanced accuracy rises correspondingly from 0.706 to 0.780 (Table 2, Figure 2). On the larger and harder 193k/87-type atlas the absolute scores are lower (87 classes is a harder problem) but

Table 2: Main result at $k=10$ (held-out macro-F1, mean $\pm$ s.d., 3 seeds). “sup.” = supervised-Wilcoxon gene selection.

Atlas	HVG	Frozen	Joint	Gain
TS-120k/59	disp.	0.730 ± 0.005	0.783 ± 0.003	$+5.2 pp$
TS-120k/59	sup.	0.740 ± 0.013	0.799	$+5.9 pp$
TS-193k/87	disp.	0.650 ± 0.006	0.752	$+10.2 pp$

the *gain grows* to $+10.2 pp$ ($0.650 \rightarrow 0.752$). The effect is largest exactly where annotation is hardest and most valuable: rare cell-type F1 improves by $+6.2 pp$ on the 120k atlas and by $+13.1 pp$ ($0.403 \rightarrow 0.534$) on the 193k atlas.

Not an artifact of weak gene selection. A natural objection is that the frozen baseline is handicapped by unsupervised (dispersion) gene selection. We therefore strengthen it with *supervised* highly-variable-gene selection—a one-vs-rest Wilcoxon test that uses the training labels to pick the most class-discriminative genes—for *both* arms. The joint projection still wins, by $+5.9 pp$ (Table 2), marginally more than against dispersion selection. Learning the projection directions helps orthogonally to how genes are selected.

4.3 The gain is regime-dependent

Figure 3 is our central result. Sweeping the number of components, the joint advantage is largest under aggressive compression and shrinks monotonically as k grows: from $+18.4 pp$ at $k=5$ (frozen 0.565 ± 0.011 , joint 0.749 ± 0.008), through $+5.2 pp$ at $k=10$ and $+2.7 pp$ at $k=20$, to only $+1.6 pp$ at $k=50$ (frozen 0.824 ± 0.011 , joint 0.841 ± 0.004), whose spread includes zero (one of three seeds marginally negative)—not significant. The mechanism is intuitive: at small k the projection must discard most of the signal, so *which* directions it keeps matters and coupling them to the task pays off; at large k the variance-maximizing basis already retains nearly all task-relevant structure, leaving little to gain. Joint optimization thus *helps when one must compress hard—the common regime for atlas-scale, memory-bound, or interpretable low-dimensional embeddings—and is harmless otherwise*. The differentiable soft- k operator, given a pool of 50 components, learns to keep an effective ~ 40 and reaches 0.830, matching the best grid value without a search.

4.4 The effect generalizes across modalities and machineries

The single-cell result is one instance of a general phenomenon. Wherever a pipeline compresses its data through a frozen, task-agnostic reduction before a model, the same substitution—initialize a learnable reduction at

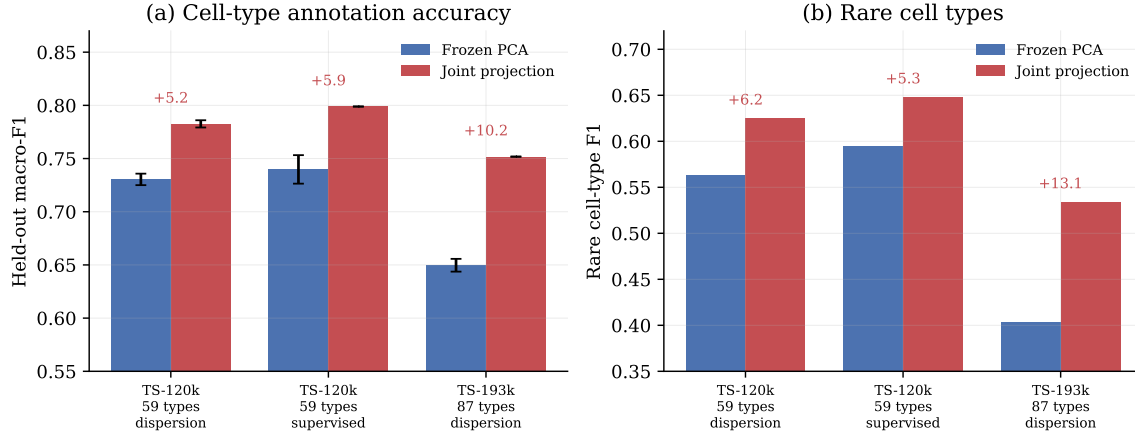


Figure 2: Joint projection versus frozen PCA at $k=10$ across atlases and gene-selection methods (mean $\pm$ s.d., 3 seeds). (a) Overall held-out macro-F1; annotations give the gain in percentage points. (b) F1 on rare cell types. The joint projection wins in every condition, by more on the larger/harder 193k/87-type atlas, and most on rare cell types.

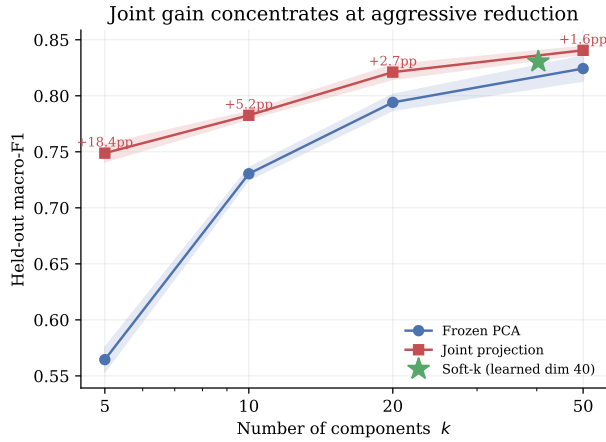


Figure 3: The joint gain concentrates at aggressive reduction. Held-out macro-F1 of frozen PCA and the joint projection as a function of the number of components k (TS-120k, mean $\pm$ s.d.). The advantage is large at small k and narrows as k grows and the frozen basis already retains near-sufficient signal. The differentiable soft- k operator ($\star$) learns an effective dimensionality of ~ 40 and matches the best fixed k without a grid search.

the frozen one and train it jointly—should recover discarded task signal, most at aggressive compression. We test this on four further pipelines spanning distinct data modalities and two different reduction *machineries*, always with the identical two-arm protocol, real data, and held-out evaluation (Table 3, Figure 4).

Four more modalities. *DNA regulatory elements*: classifying open-chromatin regions from sequence (Genomic Benchmarks [14], 174k sequences) with a frozen k -mer-spectrum-plus-PCA frontend, the joint projection gains +7.5 pp at $k=5$ —and its five learned dimensions exceed the standard 6-mer + linear-classifier baseline (logis-

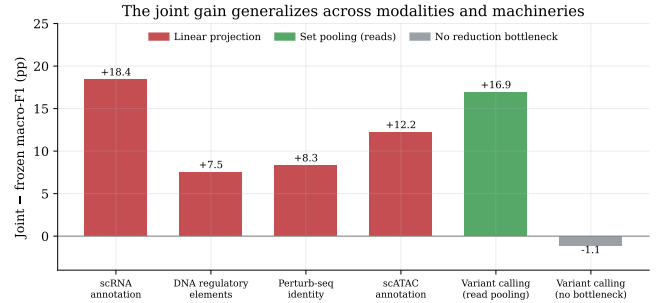


Figure 4: The joint gain generalizes. Held-out macro-F1 improvement of the jointly optimized reduction over the frozen one at aggressive compression ($k=5$), across five pipelines and two machineries. The grey bar is the no-reduction-bottleneck control on variant data, where the effect vanishes.

tic regression 0.663, linear SVM 0.697; joint 0.711). *Perturbation identity*: recovering which of 151 genetic perturbations a cell received (a Perturb-seq atlas, target-gene-masked and split by experimental batch), a deliberately hard 151-class task, it gains +8.3 pp (0.031 $\rightarrow$ 0.113 macro-F1, a 3.7 $\times$ relative improvement). *scATAC-seq annotation*: cell-type annotation of single-cell chromatin accessibility (CATLAS [15], 154k cells, 36 types) with the standard Signac TF-IDF plus latent-semantic-indexing (LSI) frontend [16]—a genuinely different frozen reduction than PCA—it gains +12.2 pp. In all four the gain-vs- k signature recurs: large under aggressive compression, shrinking as k grows.

scATAC in depth: a domain baseline and scaling. As scATAC admits a well-established annotation pipeline, we characterize it further. First, our frozen arm is a faithful baseline, not a strawman: the standard Signac workflow—LSI at 50 components followed by label transfer—reaches macro-F1 0.790 with logistic regres-

Table 3: The joint gain across pipelines at aggressive reduction ($k=5$; held-out macro-F1, mean over 3 seeds, 5 for the no-bottleneck control). Four modalities and two reduction machineries; the final row is a no-reduction-bottleneck control on the *same* variant data.

Pipeline	Modality	Machinery	Frozen macro-F1	Joint macro-F1	Gain
scRNA annotation	expression	linear projection	0.565	0.749	+18.4 <i>pp</i>
DNA regulatory elements	DNA sequence	linear projection	0.636	0.711	+7.5 <i>pp</i>
Perturb-seq identity	perturbation	linear projection	0.031	0.113	+8.3 <i>pp</i>
scATAC annotation	chromatin	linear projection (LSI)	0.476	0.598	+12.2 <i>pp</i>
Variant calling	aligned reads	read pooling (set)	0.714	0.883	+16.9 <i>pp</i>
<i>Variant calling (no bottleneck)</i>	aligned reads	pileup image + CNN	0.877	0.867	−1.1 <i>pp</i>

sion and 0.695 with k -nearest-neighbors, bracketing our frozen MLP (0.767 at $k=50$), so the joint gain is measured against competitive standard practice. Second, the gain scales with data exactly as in single cells: sweeping the training-set size at $k=5$, the joint–frozen gap runs -7.1 *pp* (5k cells), -2.7 *pp* (12k), $+4.3$ *pp* (30k), $+7.5$ *pp* (60k) to $+12.2$ *pp* (108k), crossing from negative to positive near ~ 20 k cells—a second, independent crossover that replicates the single-cell one (Figure 5a). Third, the gain is *initialization-robust*: a *randomly*-initialized learnable projection reaches the same 0.60 (vs. 0.59 from the LSI init), so it is learning the reduction, not inheriting the LSI subspace, and the uncompressed ceiling—all 12,000 TF-IDF features with no reduction—is 0.749. Finally, the whole result *replicates on a disjoint set of 16 CATLAS donors* (127k cells, 41 types): $+8.1$ *pp* at $k=5$ and $+2.3$ *pp* at $k=10$, the same signature on independent data.

Leakage and overfitting audit. A stricter audit removes two conveniences of the headline split: it holds out whole *donors* (never seen in training) and selects the top bins on the training donors only. The gain not only survives but grows—frozen macro-F1 falls to 0.39 (a task-agnostic LSI transfers poorly across donors) while the learnable projection reaches 0.53 ($+14$ *pp* at $k=5$)—so it is not a leakage artifact. This harder split does expose overfitting: with 12,000 features and 59,000 training cells the learnable projection memorizes the training donors (train F1 ≈ 1.0 vs. 0.53 held out), while the fixed-basis probe cannot (train 0.53). Stronger weight decay narrows the gap and *raises* held-out F1 ($0.49 \rightarrow 0.53$), and the orthonormal Stiefel projection generalizes best. The effect is scale-dependent: on the lower-dimensional DNA (2,080 features, 119k cells) and Perturb-seq (2,000 features, 129k cells) pipelines a three-way train/validation/test split shows the joint arm overfitting only mildly (train–test gap ≤ 6 *pp*, validation $\approx$ test) and the frozen arm not at all, with the gain intact on both held-out sets ($+7.4$ and $+7.7$ *pp*). All preprocessing throughout is fit on the training split alone.

A different machinery: pooling read evidence. The pipelines above all learn a linear projection. To test whether the effect is specific to that machinery we turn

to variant calling, where evidence is naturally reduced by pooling across sequencing reads. From real GIAB HG002 chromosome-20 alignments [17] we extract candidate sites (true variants vs. sequencing-error look-alikes), featurize each read, reduce the per-read features to k dimensions and *mean-pool over reads*—a permutation-invariant set operation—before a classifier. Replacing the frozen per-read PCA with a jointly-trained projection initialized from it improves macro-F1 by $+16.9$ *pp* at $k=5$ ($0.714 \rightarrow 0.883$), robustly across seeds. The variance-maximizing PCA never places the low-variance but task-critical signal (a mismatch at the candidate position) in its top components, so the frozen arm sits near 0.72 across $k=3-20$ while the learnable arm recovers it at once. The effect is not tied to linear-projection machinery.

A mechanistic null: no bottleneck, no gain. The win requires a genuine lossy reduction *on the gradient path*, not merely a learnable hand-designed featurization. On the same variant data we also made the DeepVariant-style pileup *image* learnable—its per-pixel channel encodings (base, quality, strand, mismatch) trainable and initialized to reproduce the fixed encoding—feeding a convolutional classifier. Here nothing is compressed: the network sees every read. Making the encoding learnable yields no improvement (-1.1 *pp*, within seed noise, with higher variance from the non-stationary frontend). Same data and the same “many learnable parameters plus a complex model,” but no reduction bottleneck, and the effect disappears—the control that pins the mechanism to compression.

4.5 Attribution and scale

The projection carries the gain. We ablate which learnable component is responsible by training only one knob at a time (Figure 5b). Making normalization or gene-weighting learnable leaves macro-F1 at the frozen baseline ($0.736 \rightarrow 0.734/0.735$); making the *projection* learnable recovers the full gain (0.765), and adding the other two on top does not improve further. The effect is specifically about learning the reduction directions, consistent with the LEAF/DiffPrep finding that the win lives in the learnable feature extractor. The attribution

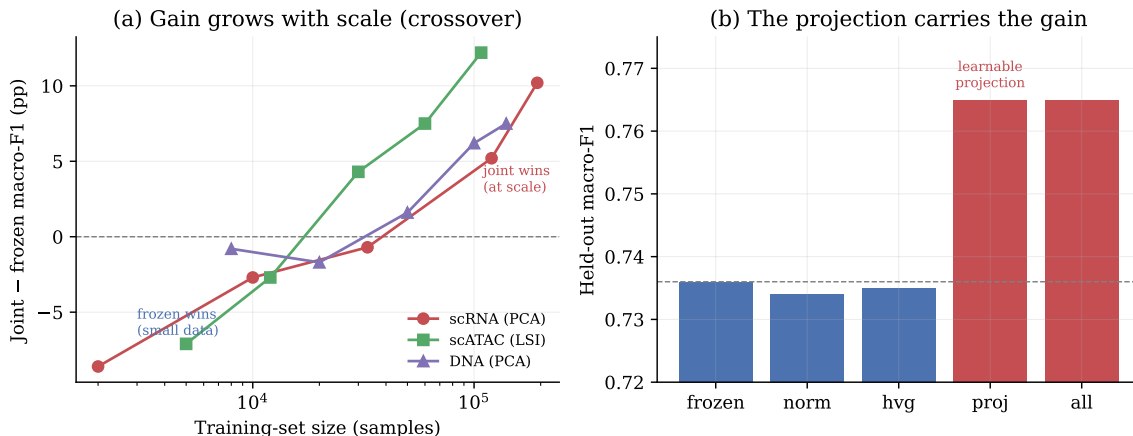


Figure 5: (a) The joint-frozen macro-F1 gap as a function of training-set size, for three modalities: negative on small data, crossing zero near ~ 20 – 40 k samples and rising to $+10.2$ *pp* (scRNA), $+12.2$ *pp* (scATAC), and $+7.5$ *pp* (DNA) at scale—an independent crossover in each. (b) Knob-attribution ablation (TS-120k, $k=10$): making normalization or gene-weighting learnable adds nothing over frozen; the learnable *projection* carries essentially the entire gain.

replicates on the Perturb-seq task ($k=10$): the learnable projection recovers $+8.2$ of the $+8.3$ *pp* total gain, while a learnable per-gene gate ($+2.7$ *pp*) and affine scaling ($+3.1$ *pp*) contribute little on their own and do not stack.

The gain grows with scale. Figure 5a plots the gap against dataset size. On small data joint optimization *hurts* (the frozen basis is a strong low-variance regularizer): the gap is -8.6 *pp* at 2,000 cells and remains negative through ~ 30 k cells, crossing zero and turning decisively positive only at scale. The scATAC and DNA pipelines show the same crossover independently (scATAC -7.1 *pp* at 5k to $+12.2$ *pp* at 108k; DNA -0.8 *pp* at 8k to $+7.5$ *pp* at 140k), so this is not a single-dataset artifact (Figure 5a). This is the predicted win-condition—large data plus aggressive reduction—and it doubles as an honest negative result (below).

4.6 Trainable and orthonormal reduction bases

The PCA-initialized residual projection is one point in a design space the operator library now spans; two variants directly address the fixed PCA anchor it carries. *Trainable basis.* A matrix-free differentiable PCA—top- k via randomized subspace (power) iteration [18], never forming the covariance—recomputes the basis from the data each step, so gradients flow through the eigensolver to any upstream learnable preprocessing: the basis *itself* is trainable, with no fixed anchor. On the DNA task ($k=5$), pairing it with a learnable per-feature scale lifts held-out macro-F1 from 0.632 (frozen basis) to 0.669 ($+3.7$ *pp*). *Orthonormal basis.* A learnable projection kept orthonormal by a QR retraction on the Stiefel manifold—no eigensolver, so no eigengap instability—reaches 0.710, match-

ing the unconstrained learnable projection (0.711) at a third of its seed variance: on this task orthonormality is essentially free. Both solvers stay finite where a differentiated dense eigendecomposition would diverge [19], realizing the trainable-basis extension at the cost of a few matmuls and a QR.

4.7 When joint optimization does not help

Three regimes eliminate the advantage, and we report them plainly. *Small data:* below ~ 30 k cells the learnable projection overfits and underperforms frozen PCA (Figure 5a); the frozen basis is a useful regularizer when labels are scarce. *Generous reduction:* at $k=50$ the gain is a non-significant $+1.6$ *pp* (Figure 3). *No reduction bottleneck:* when the frozen stage merely reformats the data rather than compressing it—the learnable-pileup-image control of Section 4.4, where the model sees every read—making it learnable does not help (-1.1 *pp*), even with many trainable parameters and a complex head. The method is therefore not a universal replacement for frozen preprocessing but a targeted improvement for the specific regime of large data reduced through an aggressive, lossy bottleneck.

5 Discussion and limitations

Our study establishes the joint-optimization effect carefully but within a bounded scope, and several limitations qualify it. (i) *Atlas scope.* Both subsets derive from a single atlas (Tabula Sapiens); although they span disjoint tissue sets and different numbers of cell types, replication on an independent, differently-sequenced atlas (e.g. HLCA or a CELLxGENE census slice) is needed to rule out protocol-specific effects.

(ii) *Baselines*. We isolate the reduction against frozen PCA; a full comparison against jointly-trained generative models (scANVI) and other supervised feature-extraction schemes remains future work. (iii) *Depth vs. breadth of the cross-modality evidence*. The single-cell annotation result is characterized in depth (atlas-size scaling, knob attribution, supervised-selection control, soft- k); scATAC now adds a scaling curve, a domain-standard baseline, initialization/no-reduction controls, and a disjoint-donor replication; DNA adds a scaling curve and a domain baseline; and Perturb-seq adds the knob attribution (Section 4.4). The main remaining gaps are the variant pipeline’s own depth—a comparison against a domain caller such as DeepVariant—and independent-dataset replication of the DNA, Perturb-seq, and variant results. (iv) *Cost*. Joint training is more expensive than fitting frozen PCA once; freezing the preprocessing statistics keeps the overhead modest (only the projection and classifier are differentiated), but it is nonzero. (v) *Uncertainty*. We report three seeds and note per-seed signs; larger seed counts with paired significance tests would sharpen the $k=50$ boundary.

6 Future work

Several directions follow directly. *Independent replication and scale*: extend the mini-batch, fit-once design to 1M+ cell atlases (HLCA, CELLxGENE) via streaming covariance estimation, testing whether the gain continues to grow. *Stronger baselines*: benchmark against scANVI and supervised metric-learning embeddings, and add a random-projection and no-reduction control. *Trainable-basis reductions*: the matrix-free differentiable PCA and Stiefel orthonormal projection of Section 4.6 are now in the library and validated on one task; scaling them to the atlas-size joint-optimization experiments—replacing the fixed anchor entirely while retaining stability [19]—is the natural next step. *Deepen the cross-modality results*: with scATAC now characterized in depth (scaling, domain baseline, controls, disjoint-donor replication), the priorities are the variant pipeline’s own depth—a domain-caller comparison such as DeepVariant—independent-dataset replication of the DNA, Perturb-seq, and variant results, and further reduction machineries (soft alignment, hidden-Markov decoding). *Automated pipeline search*: lift the fixed operator ordering into a differentiable pipeline search in the spirit of DiffPrep [4], using DiffBio’s uniform operator interface.

7 Availability and reproducibility

DiffBio is open source (MIT license) and built on JAX [8] and Flax NNX [9]. All operators, the joint-training utilities, and the benchmark harness used here ship with the library, with unit tests and fixed random seeds; the figures in this paper are regenerated from the recorded

benchmark outputs by a single script, and every cross-modality result—including the leakage and overfitting audits—is reproduced by a documented script under `benchmarks/crossmodality/`.

Every experiment fits *all* preprocessing (normalization, gene/bin selection, PCA/LSI, and the linear baselines) on the **training split only** and reports held-out metrics averaged over at least three seeds. Splits are: stratified cells for scRNA (Tabula Sapiens, 80/20); the official disjoint train/test sequences for DNA (Genomic Benchmarks); held-out experimental *batches* for Perturb-seq (target-gene columns masked); stratified cells for the headline scATAC result and held-out *donors* with train-only bin selection for its leak-free audit; and disjoint *genomic regions* for variant calling (chr20:2–22 Mb train, 30–40 Mb test). Where an experiment reports a validation set, it is carved from the training split before any fitting.

References

- [1] F Alexander Wolf, Philipp Angerer, and Fabian J Theis. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biology*, 19(1):15, 2018. doi: 10.1186/s13059-017-1382-0.
- [2] Neil Zeghidour, Olivier Teboul, Félix de Chaumont Quiry, and Marco Tagliasacchi. LEAF: A learnable frontend for audio classification. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2101.08596.
- [3] Mirco Ravanelli and Yoshua Bengio. Speaker recognition from raw waveform with SincNet. *IEEE Spoken Language Technology Workshop (SLT)*, 2018. arXiv:1812.05920.
- [4] Peng Li, Zhiyi Chen, Xu Chu, and Kexin Rong. DiffPrep: Differentiable data preprocessing pipeline search for learning over tabular data. *Proceedings of the ACM on Management of Data (SIGMOD)*, 1(2):1–26, 2023. doi: 10.1145/3589328.
- [5] Ekin D Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V Le. AutoAugment: Learning augmentation strategies from data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [6] Romain Lopez, Jeffrey Regier, Michael B Cole, Michael I Jordan, and Nir Yosef. Deep generative modeling for single-cell transcriptomics. *Nature Methods*, 15(12):1053–1058, 2018. doi: 10.1038/s41592-018-0229-2.
- [7] Chenling Xu, Romain Lopez, Edouard Mehlman, Jeffrey Regier, Michael I Jordan, and Nir Yosef. Probabilistic harmonization and annotation of single-cell transcriptomics data with deep generative models. *Molecular Systems Biology*, 17(1):e9620, 2021. doi: 10.15252/msb.20209620.
- [8] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas,

- Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs. <http://github.com/google/jax>, 2018.
- [9] Jonathan Heek, Anselm Levskaya, Avital Oliver, Marvin Ritter, Bertrand Rondepierre, Andreas Steiner, and Marc van Zee. Flax: A neural network library and ecosystem for JAX. <http://github.com/google/flax>, 2023.
 - [10] Temple F Smith and Michael S Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1):195–197, 1981.
 - [11] Ryan Poplin, Pi-Chuan Chang, David Alexander, et al. A universal SNP and small-indel variant caller using deep neural networks. *Nature Biotechnology*, 36(10):983–987, 2018. doi: 10.1038/nbt.4235.
 - [12] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015. arXiv:1502.03167.
 - [13] The Tabula Sapiens Consortium. The Tabula Sapiens: A multiple-organ, single-cell transcriptomic atlas of humans. *Science*, 376(6594):eabl4896, 2022. doi: 10.1126/science.abl4896.
 - [14] Katarina Gřešová, Vlastimil Martinek, David Čechák, Petr Šimeček, and Panagiotis Alexiou. Genomic benchmarks: a collection of datasets for genomic sequence classification. *BMC Genomic Data*, 24(1):25, 2023. doi: 10.1186/s12863-023-01123-8.
 - [15] Kai Zhang, James D Hocker, Michael Miller, Xiaomeng Hou, Joshua Chiou, Olivier B Poirion, Yunjiang Qiu, Yang Eric Li, Kyle J Gaulton, Allen Wang, Sebastian Preissl, and Bing Ren. A single-cell atlas of chromatin accessibility in the human genome. *Cell*, 184(24):5985–6001, 2021. doi: 10.1016/j.cell.2021.10.024.
 - [16] Tim Stuart, Avi Srivastava, Shaista Madad, Caleb A Lareau, and Rahul Satija. Single-cell chromatin state analysis with signac. *Nature Methods*, 18(11):1333–1341, 2021. doi: 10.1038/s41592-021-01282-5.
 - [17] Justin M Zook, Jennifer McDaniel, Nathan D Olson, Justin Wagner, Hemang Parikh, Haynes Heaton, Sean A Irvine, Len Trigg, Rebecca Truty, Cory Y McLean, Francisco M De La Vega, Chunlin Xiao, Stephen Sherry, and Marc Salit. An open resource for accurately benchmarking small variant and reference calls. *Nature Biotechnology*, 37(5):561–566, 2019. doi: 10.1038/s41587-019-0074-6.
 - [18] Nathan Halko, Per-Gunnar Martinsson, and Joel A Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011. doi: 10.1137/090771806.
 - [19] Wei Wang, Zheng Dang, Yinlin Hu, Pascal Fua, and Mathieu Salzmann. Backpropagation-friendly eigendecomposition. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. arXiv:1906.09023.